

Task 1

60 секунд			
Использование	Скорость	Базовая скорость:	2,50 ГГц
6%	1,75 ГГц	Сокетов:	1
Процессы	Потоки	Ядра:	8
259	3776	Логических процессоров:	12
Дескрипторы	117305	Виртуализация:	Включено
Время работы		Кэш L1:	704 КБ
0:01:00:13		Кэш L2:	7,0 МБ
		Кэш L3:	12,0 МБ

Hardware Property	Local Measured Value	Hardware Property	Local Measured Value
CPU Model & Architecture	12th Gen Intel(R) Core(TM) i5-12450H (2.50 GHz)	Base & Max Boost Clock	2.50GHz/2.50GHz
Physical Hardware Cores	8	Logical Cores (SMT/Threads)	12
L1 Data / Instruction Cache	704kb	L2 Cache (Per Core / Total)	7Mb
L3 Shared Last-Level Cache	12mb	SIMD Vector Extensions	AVX2 (Advanced Vector Extensions 2.0) AVX SSE4.1 / SSE4.2 SSSE3 / SSE3 / SSE2 / SSE FMA3 (Fused Multiply-Add)

Q1.1: Map your host machine's execution modes into Flynn's Taxonomy:

SISD: The CPU operates in SISD mode during single-threaded, sequential tasks like a compiler pass or standard CLI utilities execution.

SIMD: It switches to SIMD for data-parallel vector math (e.g., matrix multiplication) by leveraging hardware-accelerated **AVX2** and **SSE4.2** instructions.

MIMD: The machine acts as a MIMD system during multi-core OS scheduling and multi-threaded applications (like your Java benchmark), where multiple independent cores execute distinct instruction streams on different data simultaneously.

Task 2

Java code :

```
public class Thread_test {

    public static void main(String[] args) throws InterruptedException {
        System.out.println("== Starting Multi-Thread Scaling Benchmark ==");
        System.out.println("CPU: Intel Core i5-12450H (4P + 4E Cores, 12 Threads)");
        System.out.println("Workload: Prime Check up to " + LIMIT + "\n");
        System.out.printf("%-10s %-10s %-10s %-10s %-12s\n", "Threads(N)", "Run 1 (s)", "Run 2 (s)", "Run 3 (s)", "Avg (s)");
        System.out.println("-----");

        // Прогрев JVM (JIT compiler warm-up)
        runWorkload( threadCount: 1);

        for (int threads : THREADS_CONFIG) {
            double[] runs = new double[ITERATIONS];
            double sum = 0;

            for (int i = 0; i < ITERATIONS; i++) {
                long startTime = System.nanoTime();
                runWorkload(threads);
                long endTime = System.nanoTime();

                runs[i] = (endTime - startTime) / 1_000_000_000.0;
                sum += runs[i];
            }
            double avg = sum / ITERATIONS;
            System.out.printf("%-10d %-10.3f %-10.3f %-10.3f %-12.3f\n", threads, runs[0], runs[1], runs[2], avg)
        }
    }
}
```

The result :

```

=== Starting Multi-Thread Scaling Benchmark ===
CPU: Intel Core i5-12450H (4P + 4E Cores, 12 Threads)
Workload: Prime Check up to 5000000

```

```

Threads(N) Run 1 (s)  Run 2 (s)  Run 3 (s)  Avg (s)
-----
1          0,627      0,546      0,538      0,571
2          0,396      0,398      0,428      0,407
4          0,298      0,338      0,425      0,354
8          0,148      0,151      0,148      0,149
16         0,145      0,129      0,125      0,133
32         0,108      0,118      0,116      0,114

```

```

Process finished with exit code 0

```

Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time TN (s)	Speedup SN = T1/TN	Efficiency EN = SN/N
N = 1 (Baseline)	0.627	0.546	0.538	0.571	1.00×	100.0%
N = 2	0.396	0.398	0.428	0.407	1.40×	70.0%
N = 4	0.338	0.338	0.425	0.354	1.61×	40.3%
N = 8	0.148	0.151	0.148	0.149	3.83×	47.9%
N = 16	0.145	0.129	0.125	0.133	4.29×	26.8%
N = 32	0.108	0.118	0.116	0.114	5.01×	15.7%

Q2.1: Scaling Saturation & Hardware Contention Analysis:

The sub-linear saturation begins at $N = 2$ due to the unequal performance of P and E cores, but the definitive hardware wall is hit at $N = 8$. Beyond 8 threads, the workload exceeds the CPU's 8 physical cores, forcing heavy compute-bound tasks to share ALU units via Hyper-Threading with minimal gain [12th Gen Intel(R) Core(TM) i5-12450H]. At $N = 16$ and 32, scaling completely flattens because the thread count exceeds the 12 logical processors, causing massive OS context-switching overhead.

Task 3:

Code result :

```
Run #1 | Output: 4731051 | Error: 5268949
Run #2 | Output: 9307221 | Error: 692779
Run #3 | Output: 10000000 | Error: 0
Run #4 | Output: 10000000 | Error: 0
Run #5 | Output: 10000000 | Error: 0
Run #6 | Output: 10000000 | Error: 0
Run #7 | Output: 10000000 | Error: 0
Run #8 | Output: 10000000 | Error: 0
Run #9 | Output: 10000000 | Error: 0
Run #10 | Output: 10000000 | Error: 0

Process finished with exit code 0
```

Run	Measured Output	Error (10^7 - Act)	Run	Measured Output	Error (10^7 - Act)
#1	4731051	5268949	#6	10000000	0
#2	9307221	692779	#7	10000000	0
#3	10000000	0	#8	10000000	0
#4	10000000	0	#9	10000000	0
#5	10000000	0	#10	10000000	0

Q3.1: Read-Modify-Write

Race condition occurs because `counter++` consists of three operations:

Load → Add → Store

Two cores can load the same value before either stores the result. One update then overwrites the other, causing lost increments and an incorrect final value.

Q3.2: Synchronization Penalty

Unlocked = ____ ms vs. Locked = ____ ms

Locked code is slower because threads must **wait for the lock** before accessing the counter. Lock acquisition/release and thread contention add overhead.

Task 4

1) $T_1 = 0.571 \text{ s}$ | $T_2 = 0.407 \text{ s}$ / Calculated $p = 0.574$

2) Amdahl Theoretical Speedup Ceiling

$$1-p = 1 - 0.5744 = 0.4256$$

$$S = 1 / 0.4256 = 2.35$$

3) Amdahl 64-Core Projection

$$S = 1 / (0.4256 + 0.5744 / 64) \approx 2.33$$

4) Gustafson Scaled Speedup (Weak Scaling)

$$S = (1-p) + (p \times 64) = 0.4256 + (0.5744 \times 64) \approx 37.19$$

Q4.1: Strong vs. Weak Scaling Analysis

Strong scaling: The problem size stays fixed while the number of threads increases. In my results, performance improved from **0.571 s with 1 thread to 0.114 s with 32 threads**, showing good speedup, but efficiency decreased as thread count increased.

Weak scaling: The problem size increases with the number of threads. According to Gustafson's Law, a 64× larger problem could achieve a theoretical speedup of approximately **37.19×**, because more of the workload can be executed in parallel.